

A comunidade de economia circular do teu campus

Unico

Documentação da Base de Dados

Modelação, modelo relacional, dicionário de dados e decisões de design

Unidade curricular	Sistemas de Gestão de Base de Dados (SGBD)
Projeto	Projeto Interdisciplinar PI1 · DTAM 2025/2026
Instituição	ESMAD – Escola Superior de Media, Artes e Design · P.PORTO
Grupo	Grupo 1
Elementos	Vítor Lima · Nuno Tiago
Motor	PostgreSQL 17 (Supabase) · esquema public · 8 tabelas

1. Introdução

Este documento descreve a base de dados da aplicação **UniCo**, um *marketplace* de economia circular para a comunidade de um campus universitário, onde os estudantes compram, vendem e trocam artigos em segunda mão. A base de dados está implementada em **PostgreSQL**, alojada na plataforma **Supabase**, que disponibiliza ainda a autenticação dos utilizadores e o armazenamento das imagens.

O documento apresenta o modelo conceptual (diagrama entidade-relacionamento), o modelo relacional com as chaves primárias e estrangeiras, o dicionário de dados de cada tabela e as decisões de modelação, incluindo a análise de normalização e as políticas de segurança ao nível da linha (Row Level Security). O esquema é composto por **8 tabelas** no esquema public. O script completo que recria toda a estrutura encontra-se no ficheiro **g1_base_dados.sql**.

2. Diagrama Entidade-Relacionamento (modelo conceptual)

O diagrama seguinte mostra as oito entidades e as relações entre elas. A notação pé-de-galinha indica a cardinalidade: o lado **||** representa “exatamente um” e o lado **<0** representa “zero ou muitos”, ou seja, relações um-para-muitos (1:N). Cada tabela indica a chave primária (PK), a chave única (UK) e as chaves estrangeiras (FK). A entidade utilizador estende a tabela auth.users do Supabase, partilhando o mesmo id.

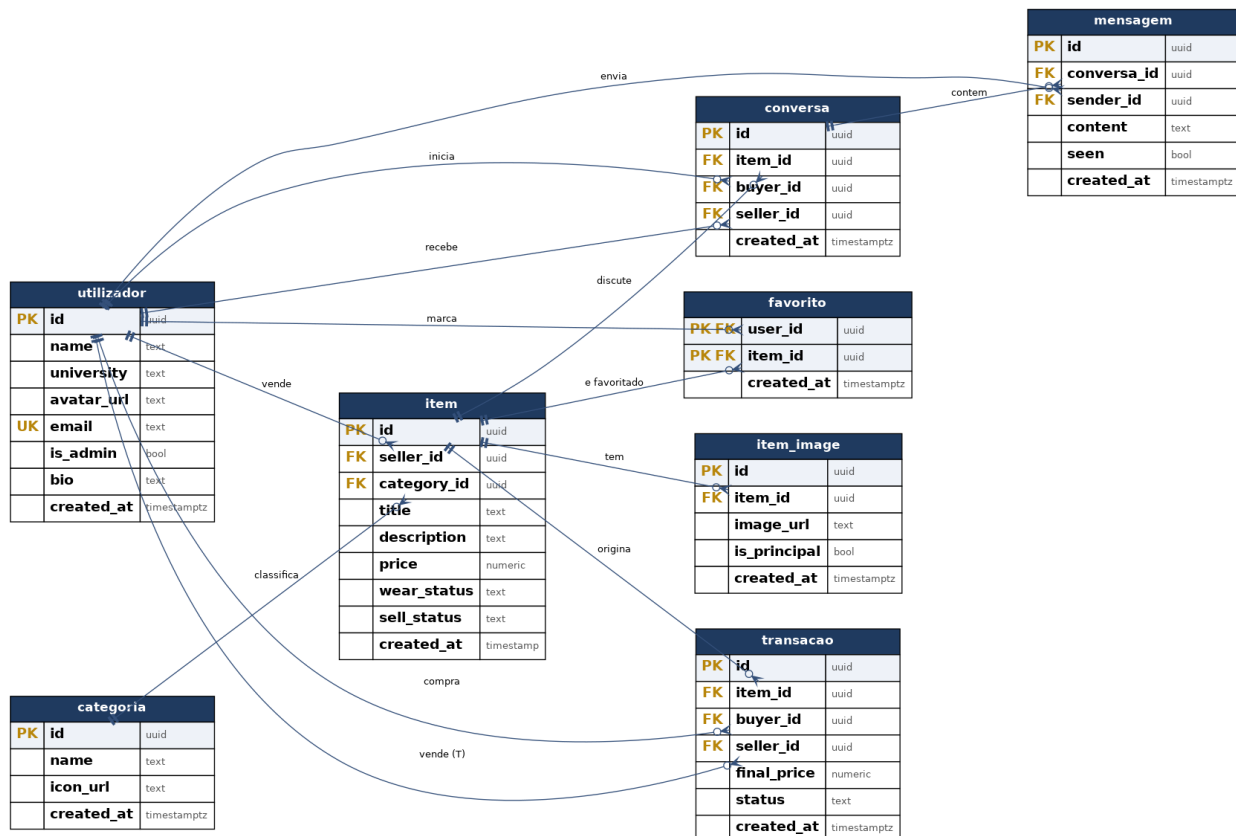


Figura 1 — Diagrama entidade-relacionamento da base de dados UniCo.

3. Modelo relacional

Notação: a chave primária está sublinhada; as chaves estrangeiras (FK) estão listadas por baixo de cada relação com a referência → tabela(coluna).

utilizador(id, name, university, avatar_url, email, is_admin, bio, created_at)

FK: id → auth.users(id)

categoria(id, name, icon_url, created_at)

Sem chaves estrangeiras.

item(id, seller_id, category_id, title, description, price, wear_status, sell_status, created_at)

FK: seller_id → utilizador(id); category_id → categoria(id)

item_image(id, item_id, image_url, is_principal, created_at)

FK: item_id → item(id)

favorito(user_id, item_id, created_at)

PK composta (N:M). FK: user_id → utilizador(id); item_id → item(id)

transacao(id, item_id, buyer_id, seller_id, final_price, status, created_at)

FK: item_id → item(id); buyer_id → utilizador(id); seller_id → utilizador(id)

conversa(id, item_id, buyer_id, seller_id, created_at)

FK: item_id → item(id); buyer_id → utilizador(id); seller_id → utilizador(id). UNIQUE(item_id, buyer_id, seller_id)

mensagem(id, conversa_id, sender_id, content, seen, created_at)

FK: conversa_id → conversa(id); sender_id → utilizador(id)

Cardinalidade das relações

Relação	Card.	Descrição
utilizador – item	1:N	Um utilizador publica vários itens (seller_id).
categoria – item	1:N	Uma categoria classifica vários itens.
item – item_image	1:N	Um item tem várias imagens (até 5).
utilizador – item (favorito)	N:M	Resolvida pela tabela de junção favorito.
item – transacao	1:N	Um item pode originar transações.
utilizador – transacao	1:N	Utilizador como comprador e como vendedor.
item – conversa	1:N	Um item gera várias conversas de negociação.
conversa – mensagem	1:N	Uma conversa contém várias mensagens.

4. Dicionário de dados

Para cada tabela apresentam-se os campos, o tipo de dados, as restrições de integridade e uma breve descrição. Legenda das restrições: **PK** chave primária, **FK** chave estrangeira, **UK** única, **NN** obrigatório (NOT NULL).

utilizador

Campo	Tipo	Restrições	Descrição
id	uuid	PK, FK, NN	Identificador do utilizador; é o mesmo id do Supabase Auth (auth.users).
name	text	NN	Nome do utilizador.
university	text	—	Universidade/campus a que pertence.
avatar_url	text	—	URL da fotografia de perfil.
email	text	UK	Email institucional; único em toda a tabela.
is_admin	boolean	NN, def. false	Indica se o utilizador é administrador.
bio	text	—	Pequena descrição pessoal.
created_at	timestampz	NN, def. now()	Data e hora de criação do perfil.

categoria

Campo	Tipo	Restrições	Descrição
id	uuid	PK, def. gen_random_uuid()	Identificador da categoria.
name	text	NN	Nome da categoria (ex.: Manuais, Informática).
icon_url	text	—	Ícone associado à categoria.
created_at	timestampz	—	Data de criação.

item

Campo	Tipo	Restrições	Descrição
id	uuid	PK, def. gen_random_uuid()	Identificador do anúncio/artigo.
seller_id	uuid	FK, NN	Vendedor que publicou o item (utilizador).
category_id	uuid	FK	Categoria do item.
title	text	—	Título do anúncio.
description	text	—	Descrição detalhada do artigo.
price	numeric	—	Preço pedido, em euros.
wear_status	text	CHECK	Estado de conservação: novo, como_novo, bom, satisfatorio, usado, muito_usado.
sell_status	text	CHECK	Estado da venda: disponivel, reservado, vendido.
created_at	timestamp	—	Data de publicação (sem fuso horário).

item_image

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador da imagem.
item_id	uuid	FK, NN, CASCADE	Item a que a imagem pertence.
image_url	text	—	URL/caminho da imagem.
is_principal	boolean	—	Indica se é a imagem principal do anúncio.
created_at	timestampz	—	Data de carregamento da imagem.

favorito

Campo	Tipo	Restrições	Descrição
user_id	uuid	PK, FK, CASCADE	Utilizador que guardou o item (parte da PK composta).
item_id	uuid	PK, FK, CASCADE	Item guardado como favorito (parte da PK composta).
created_at	timestampz	—	Data em que o item foi guardado.

Restrição de tabela: PRIMARY KEY (user_id, item_id).

transacao

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador da transação.
item_id	uuid	FK, NN, CASCADE	Item que foi transacionado.
buyer_id	uuid	FK, NN	Comprador (utilizador).
seller_id	uuid	FK, NN	Vendedor (utilizador).
final_price	numeric	NN, CHECK >= 0	Preço final acordado para o negócio.
status	text	NN, def. 'pendente', CHECK	Estado: pendente, concluída, cancelada.
created_at	timestampz	NN, def. now()	Data e hora da transação.

conversa

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador da conversa.
item_id	uuid	FK, CASCADE	Item sobre o qual decorre a negociação.
buyer_id	uuid	FK, CASCADE	Comprador participante.
seller_id	uuid	FK, CASCADE	Vendedor participante.
created_at	timestampz	NN, def. now()	Início da conversa.

Restrição de tabela: UNIQUE (item_id, buyer_id, seller_id) — uma só conversa por trio.

mensagem

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador da mensagem.
conversa_id	uuid	FK, NN, CASCADE	Conversa a que a mensagem pertence.
sender_id	uuid	FK, CASCADE	Autor (remetente) da mensagem.
content	text	NN	Conteúdo textual da mensagem.
seen	boolean	NN, def. false	Indica se a mensagem já foi vista pelo destinatário.
created_at	timestampz	NN, def. now()	Data e hora de envio.

5. Decisões de modelação

5.1 Normalização (1FN, 2FN, 3FN)

1FN. Todos os atributos são atômicos (um valor por célula), sem grupos repetidos nem listas, e cada tabela tem chave primária. As várias imagens de um item não estão numa coluna multivalor — vivem em linhas separadas na tabela `item_image`.

2FN. Estando em 1FN, todos os atributos não-chave dependem da chave primária completa. A maioria das tabelas tem chave simples (`id`), logo não há dependências parciais. A única chave composta é `favorito` (`user_id`, `item_id`) e o seu único atributo não-chave (`created_at`) depende da combinação completa.

3FN. Estando em 2FN, não existem dependências transitivas: os dados que pertencem a outra entidade são referenciados por chave estrangeira em vez de copiados. Por exemplo, o nome da categoria vive apenas em `categoria` (o item guarda só `category_id`) e os dados do utilizador vivem apenas em `utilizador`. Conclui-se que o esquema está normalizado até à **3.ª Forma Normal**.

5.2 Relações e tabela de junção

A relação muitos-para-muitos entre utilizadores e itens (`favoritos`) é resolvida com a tabela de junção `favorito`, cuja chave primária composta (`user_id`, `item_id`) garante que o mesmo utilizador não guarda o mesmo item duas vezes. As fotografias seguem uma relação 1:N em `item_image`, e o chat segue o padrão conversa 1:N mensagem.

5.3 Integração com a autenticação

A tabela `utilizador` partilha a chave primária com `auth.users` do Supabase (relação 1:1 identificadora). Assim, o perfil aplicacional fica separado das credenciais de autenticação, mas ligado por `id`, evitando duplicação de dados de identidade.

5.4 Desnormalização controlada

As tabelas `transacao` e `conversa` guardam `seller_id`, apesar de este ser derivável de `item.seller_id`. Trata-se de redundância **intencional**: preserva quem era o vendedor à data do registo (memória histórica, mesmo que o item seja alterado ou removido) e evita um JOIN adicional nas consultas mais frequentes. É a única exceção deliberada à 3FN.

5.5 Integridade de domínio (CHECK) e restrições

Os estados são limitados por restrições CHECK em vez de texto livre: `wear_status` (6 valores), `sell_status` (disponível/reservado/vendido) e `status` da transação (pendente/concluída/cancelada). Há ainda `final_price >= 0` e a restrição UNIQUE em `conversa` para impedir conversas duplicadas. As eliminações em cascata (ON DELETE CASCADE) mantêm a coerência: apagar um item remove as suas imagens, favoritos e conversas associadas.

5.6 Segurança (Row Level Security)

O RLS está ativo nas 8 tabelas, garantindo que cada utilizador só acede ao que lhe diz respeito: os perfis e itens são de leitura pública, mas só o dono os edita; os favoritos, transações e conversas só são visíveis aos respetivos participantes; e apenas administradores gerem categorias. A função auxiliar `is_admin()` centraliza a verificação de privilégios de administrador.

5.7 Índices e tipos temporais

Além dos índices implícitos das chaves primárias e únicas, foram criados índices secundários sobre as colunas mais filtradas (`seller_id`, `category_id`, `item_id`, `user_id`, `conversa_id`) para acelerar as listagens. A maioria dos carimbos temporais usa `timestamp_tz`; `item.created_at` usa `timestamp` sem fuso, por opção da equipa no momento da criação.